Sprint 4 — Pod A: The Injection(For Thursday)

1. The GSP Injection

This is the core of the Sprint 4 work. GSP error (XID 119) is the highest-priority failure type because it has 100% job-failure probability and a 99% cascade probability. It is the single most disruptive failure in the literature, which is why it leads the injection work.

1.1 The Simulation Hook

ASTRA-sim has two executables:

- AstraSim_Analytical_Congestion_Unaware — baseline, no failure support
- AstraSim_Analytical_Congestion_Aware — **this is the one we target** — it models link states, and a failed node is implemented as a link/node marked unavailable mid-run

The injection hook is where the simulation marks a node as unavailable.

1.2 GSP Error — Background

GSP (GPU System Processor) is the firmware controller that manages GPU health, power, memory, and error handling. When it fails:

- The GPU becomes completely inoperable — it cannot compute, communicate, or receive commands
- The failure is detected almost instantly (0.2 seconds propagation time to inoperable state)
- Recovery requires a full node reboot, which takes anywhere from 0.3 hours (average) to 23 hours (worst case documented in the literature)
- It correlates with demanding ML workloads — our LLM simulation may produce GSP faults more often than the average MTBE suggests

Why it leads:

Failure Type	MTBE (node-hrs)	Job Fail %	Cascade %	Why it leads
GSP Error (XID 119)	**39.55**	**100%**	**99%**	Every GSP fault kills the job and almost certainly cascades — highest-impact single failure type
NVLink Error (XID 74)	28.28	65.7%	16% multi-GPU	Frequent but partial — most stay on one GPU (84%)
Uncontained ECC (XID 95)	2.17	97.2%	None — isolated	Very frequent but contained — no cascade; single GPU reset

1.3 The Injection Event Sequence

Each step is a discrete event the simulation executes in order.

Step 1 — Trigger condition (when does the fault fire?)

The fault fires mid-training, during the iterative training phase. Specifically:

- Trigger time: after at least one training step has completed cleanly (so TC-01 and TC-02 baselines are established first)
- Trigger mechanism: MTBE-based probabilistic injection — at each epoch boundary, sample a Poisson process with rate $\lambda = 1 / 39.55$ node-hrs to decide whether a GSP fault fires on each node

> *For Thursday: inject one GSP fault at a fixed time (e.g. after step 3 of 10). This gives a clean before/after JCT comparison without needing the full probabilistic harness.*

Step 2 — Immediate effect (what happens at fault time T_0?)

- Target node: one NPU, chosen randomly (or fixed for the demo — NPU 0 is fine)
- At T_0 + 0.2 seconds simulated time: mark the target node as inoperable
- 'Inoperable' means: cannot execute compute operations, cannot participate in collective communication, does not respond to any further simulation events
- All in-flight collective operations involving this node enter stall state — this is the NCCL stall that Pod B harness detects

Step 3 — Cascade branch (probabilistic, 99% path)

After the node is marked inoperable, apply the cascade:

- With probability 0.99: the GPU becomes permanently inoperable for this training job — trigger node reboot sequence (Step 4)
- With probability 0.01: the fault propagates to a PMU SPI error instead — see Section 4 for the cascade chain (stretch)
- For the Thursday demo: always take the 0.99 path — treat it as deterministic

Step 4 — Node reboot / recovery window

- Remove the failed node from all active process groups and collective operations
- Recovery window: sample uniformly from [0.3 hours, 23 hours] in simulated time (UIUC/IBM empirical range)
- For the Thursday demo: use a fixed recovery window of 0.3 hours (18 minutes) — the average from the paper — to keep the JCT delta clean and reproducible
- During the recovery window: the cluster continues with N-1 nodes; all-reduce rings must be reformed without the failed participant
- After recovery: the node comes back online (optional for Sprint 4 — it is fine to leave the node down for the duration of the run)

Step 5 — What gets logged (the observability contract)

Every injection event must write the following fields to the simulation log. Pod B reads these fields to compute the JCT recovery penalty.

Field	What it records	Written by	Read by
fault_node_id	Which NPU failed (e.g. NPU 0)	Pod A (Tae wires, recipe here)	Pod B — cascade tracking
fault_timestamp_T0	Simulation time when fault fires	Pod A	Pod B — stall duration clock start
recovery_window_hrs	Sampled recovery duration (hrs simulated)	Pod A	Pod B — penalty calculation
nodes_affected_pct	% of cluster affected at peak propagation	Pod A (= 1/N for isolated GSP)	Pod B — cluster affected % metric
reboot_events_per_epoch	Count of node-removal/reboot events per training epoch	Pod A	Pod B — reboot frequency metric

1.4 The Full Recipe as a Written Sequence

GSP INJECTION RECIPE — SINGLE-FAULT DEMO

```
Precondition: at least 1 training step completed cleanly (TC-01 baseline
recorded)

1. At fixed simulation timestep T_inject (e.g. after step 3 of 10):
   → Write fault_node_id = 0, fault_timestamp_T0 = T_inject to log

2. At T_inject + 0.2 sim-seconds:
   → Mark NPU 0 as INOPERABLE
   → Remove NPU 0 from all active NCCL process groups
   → All in-flight collectives involving NPU 0 enter STALL state
   → (Pod B detector fires here — hard stall threshold crossed)

3. Cascade branch (deterministic for demo — always take 0.99 path):
   → GPU permanently inoperable for this job
   → Trigger node removal

4. Recovery window:
   → Set recovery_window_hrs = 0.3 (average from UIUC/IBM paper)
   → Write recovery_window_hrs to log
   → Continue simulation with N-1 = 3 NPUs
   → Reform all-reduce ring over remaining NPUs

5. Write to observability log:
   → fault_node_id: 0
   → fault_timestamp_T0: <T_inject>
   → recovery_window_hrs: 0.3
   → nodes_affected_pct: 1/4 = 25%  (1 of 4 NPUs)
   → reboot_events_per_epoch: 1

6. Continue simulation to completion with 3 NPUs.
   → Record JCT_failure
   → Pod B computes: JCT Recovery Penalty = JCT_failure - JCT_baseline
```

That is TC-03's critical bar: one fault, cleanly injected, with the recovery penalty logged. Everything else is stretch.

2. The Observability Contract — How Pod A and Pod B Connect

The observability contract is the interface between injection (Pod A) and measurement harness (Pod B). The fields in this table are what the injection writes and what Pod B reads to compute the JCT recovery penalty.

The contract is the coordination surface.

Field	What it records	Pod A	Pod B	Verifies
JCT Recovery Penalty	(JCT with failure) − (JCT baseline)	Triggers the event	Measures the cost	FR-G2, TC-03
NCCL Stall Duration	fault_timestamp_T0 → hard-stall detection timestamp	Writes T0 to log	Detects stall; computes duration	FR-N2
Cluster Affected %	Share of nodes degraded at peak propagation	Writes nodes_affected_pct	Reads from log	FR-G3
Node Reboot Frequency	Node-removal/reboot events per epoch	Writes reboot_events_per_epoch	Reads from log	FR-G2
Routing Benefit	ECMP vs. predictive penalty gap — defined now, used Sprint 5	Not yet — Sprint 5	Not yet — Sprint 5	FR-P2, TC-04

2.1 How the Stall Detection Connects to Injection

- When we mark NPU 0 as inoperable, all in-flight AllReduce operations involving NPU 0 enter stall state
- The detector fires when a collective exceeds its expected completion time by a factor of k_hard = 3 — this is the hard stall signal
- The **NCCL Stall Duration** is measured from your fault timestamp (T0) to the hard-stall detection timestamp — you write T0, Pod B reads it
- The JCT Recovery Penalty clock starts at hard-stall detection — the earliest defensible moment the fault began to cost compute time

> *In plain terms: Pod A fire the fault and log the time. Pod B harness detects the stall and measures how long it takes. The two halves of TC-03 meet in those two timestamps.*